

บทที่ 3

วิธีดำเนินงาน

ผู้จัดทำพัฒนา Chatshop เป็น 2 ส่วน ได้แก่ แอปพลิเคชันบนสมาร์ตโฟนที่เจ้าหน้าที่ร้านค้าใช้งาน และระบบบริการหลังบ้านที่รับส่งข้อความกับแพลตฟอร์มภายนอก จัดเก็บข้อมูล และประมวลผลแชทบอท บทนี้อธิบายขั้นตอนการดำเนินงานตั้งแต่การวิเคราะห์ปัญหาจนถึงการพัฒนาทั้งสองส่วน โดยมีหัวข้อดังนี้

- 3.1 กำหนดปัญหาที่เกิดขึ้นกับระบบงาน
- 3.2 การวิเคราะห์ระบบ
- 3.3 การออกแบบระบบ
- 3.4 โครงสร้างแอปพลิเคชันบนสมาร์ตโฟน
- 3.5 โครงสร้างระบบบริการหลังบ้าน
- 3.6 การออกแบบ API
- 3.7 การพัฒนาแอปพลิเคชันบนสมาร์ตโฟน
- 3.8 การเชื่อมต่อกับ LINE Official Account และ Meta

3.1 กำหนดปัญหาที่เกิดขึ้นกับระบบงาน

ร้านค้าออนไลน์ที่ขายผ่านแชทต้องเปิดแอปพลิเคชันของแต่ละช่องทางแยกกัน เจ้าหน้าที่ต้องสลับหน้าจอไปมาจนพลาดการแจ้งเตือน เมื่อมีหลายคนช่วยกันตอบก็ตอบซ้ำกันหรือตกหล่นคำถามเดิมต้องพิมพ์ตอบซ้ำทุกวัน และการเปิดคำสั่งซื้อยังจดด้วยมือแยกจากแชท ผู้จัดทำจึงพัฒนาระบบหลังบ้านที่รับข้อความจากทุกช่องทางมาเก็บไว้ที่เดียว และพัฒนาแอปพลิเคชันที่แสดงข้อความเหล่านั้นในกล่องข้อความเดียวแบบเรียลไทม์ พร้อมการแจ้งเตือนแบบพุช แชทบอท และการเปิดคำสั่งซื้อจากหน้าต่างสนทนา

3.2 การวิเคราะห์ระบบ

ผู้จัดทำวิเคราะห์ระบบจากขั้นตอนการทำงานจริงของร้านค้าที่ขายผ่านแชท แล้วสรุปเป็นผู้ใช้งาน ความต้องการด้านการทำงาน และความต้องการด้านคุณภาพของระบบ ดังนี้

3.2.1 ผู้ใช้งานของระบบ แบ่งออกเป็น 3 กลุ่ม ดังนี้

3.2.1.1 เจ้าของร้าน ผู้สมัครใช้งาน เชื่อมต่อช่องทางแชท เชิญสมาชิกเข้าทีม กำหนดบทบาท และตั้งค่าร้านค้าและแชทบอทผ่านแอปพลิเคชัน

3.2.1.2 เจ้าหน้าที่ ได้แก่ ผู้จัดการ แอดมิน และพนักงานตอบแชท ใช้งานได้ตามสิทธิ์ของบทบาท เช่น ตอบแชท เปิดคำสั่งซื้อ หรือดูรายงาน

3.2.1.3 ลูกค้า ผู้ส่งข้อความหาร้านผ่าน LINE, Facebook Messenger, Instagram หรือ WebChat โดยไม่ต้องติดตั้งแอปพลิเคชันของร้าน

3.2.2 Functional Requirements แบ่งตามส่วนของระบบ ดังนี้

3.2.2.1 ส่วนแอปพลิเคชันบนสมาร์ตโฟน

- 1) ผู้ใช้ต้องสมัครและเข้าสู่ระบบด้วยอีเมล Facebook หรือ Google ได้
- 2) เจ้าหน้าที่ต้องดูบทสนทนาจากทุกช่องทางในกล่องข้อความเดียว เลือกดูตามช่องทาง เพจ ข้อความที่ยังไม่อ่าน และค้นหาได้
- 3) เจ้าหน้าที่ต้องตอบข้อความ ส่งรูปภาพ วิดีโอ และไฟล์เอกสาร ใช้ข้อความสำเร็จรูป และตอบกลับโดยอ้างอิงข้อความเดิมได้
- 4) แอปพลิเคชันต้องแสดงข้อความใหม่ทันทีโดยผู้ใช้ไม่ต้องรีเฟรช และแจ้งเตือนแบบพุชเมื่อไม่ได้เปิดแอปพลิเคชันอยู่
- 5) เจ้าหน้าที่ต้องมอบหมาย ปักหมุด จัดเก็บ และติดแท็กบทสนทนาได้
- 6) เจ้าหน้าที่ต้องเปลี่ยนโหมดแชทบอททราายบทสนทนา ขอคำตอบแนะนำ และดูผลวิเคราะห์บทสนทนาได้
- 7) เจ้าหน้าที่ต้องจัดการข้อมูลลูกค้าและสินค้า และเปิดคำสั่งซื้อจากหน้าต่างสนทนาได้
- 8) เจ้าของร้านต้องจัดการทีม ช่องทาง และการตั้งค่าร้านค้าได้
- 9) ผู้ใช้ต้องดูรายงานภาพรวมการให้บริการได้

3.2.2.2 ส่วนระบบบริการหลังบ้าน

- 1) ระบบต้องรับข้อความจาก LINE Official Account, Facebook Messenger, Instagram และ WebChat ผ่าน Webhook
- 2) ระบบต้องส่งข้อความที่เจ้าหน้าที่ตอบกลับออกไปยังแพลตฟอร์มของลูกค้าได้ถูกต้อง
- 3) ระบบต้องส่งเหตุการณ์ไปยังแอปพลิเคชันแบบเรียลไทม์ และส่งการแจ้งเตือนผ่าน Firebase Cloud Messaging
- 4) ระบบต้องแจ้งเตือนเจ้าหน้าที่เมื่อมีแชทค้างตอบนานเกินกำหนด
- 5) ระบบต้องสร้างคำตอบด้วยแชทบอทจากฐานความรู้ของร้าน และโอนให้เจ้าหน้าที่เมื่อตอบไม่ได้
- 6) ระบบต้องตอบกลับอัตโนมัติตามคีย์เวิร์ดที่ร้านกำหนด

7) ระบบต้องเปิด REST API ให้แอปพลิเคชัน และตรวจสอบสิทธิ์ทุกคำขอ

3.2.3 Non-Functional Requirements เป็นความต้องการด้านคุณภาพของระบบ ดังนี้

3.2.3.1 ด้านประสิทธิภาพ ระบบหลังบ้านต้องตอบกลับ Webhook ได้ทันที แล้วจึงประมวลผลข้อความในคิวงานเบื้องหลัง เพื่อไม่ให้แพลตฟอร์มส่ง Webhook ซ้ำ

3.2.3.2 ด้านความปลอดภัย ระบบต้องตรวจสอบลายเซ็นของ Webhook ทุกครั้ง ยืนยันตัวตนด้วยโทเคนก่อนเรียกใช้ API และแอปพลิเคชันต้องเก็บโทเคนในพื้นที่จัดเก็บที่เข้ารหัส

3.2.3.3 ด้านการแยกข้อมูล ข้อมูลของแต่ละร้านค้าต้องแยกออกจากกัน ผู้ใช้เห็นได้เฉพาะข้อมูลของร้านตนเอง แม้ทุกร้านจะใช้ฐานข้อมูลชุดเดียวกัน

3.2.3.4 ด้านการใช้งาน แอปพลิเคชันต้องแสดงผลการกระทำของผู้ใช้ทันที เช่น การส่งข้อความหรือการมอบหมายงาน และย้อนกลับสถานะเดิมเมื่อเครื่องแม่ข่ายตอบว่าไม่สำเร็จ

3.2.3.5 ด้านความน่าเชื่อถือ ระบบต้องลองส่งงานใหม่เมื่อล้มเหลว ต้องไม่ส่งข้อความเดิมซ้ำให้ลูกค้า และแอปพลิเคชันต้องดึงข้อมูลล่าสุดใหม่ทุกครั้งที่เชื่อมต่อเรียลไทม์กลับมาได้

3.2.4 Use Case Diagram แสดงขอบเขตของระบบและความสัมพันธ์ระหว่างผู้ใช้งานกับฟังก์ชันของระบบ ดังรูปที่ 3.1

[ใส่รูป: Use Case Diagram ของระบบ Chatshop]

รูปที่ 3.1 Use Case Diagram ของระบบ Chatshop

จากรูปที่ 3.1 ผู้ใช้งานแต่ละกลุ่มทำงานได้ ดังนี้

3.2.4.1 เจ้าของร้าน จัดการช่องทาง ทีม บทบาท การตั้งค่าร้าน และแชทบอท และใช้งานทุกฟังก์ชันของเจ้าหน้าที่ได้โดยไม่ต้องกำหนดสิทธิ์เพิ่ม

3.2.4.2 เจ้าหน้าที่ ตอบแชท จัดการบทสนทนา ลูกค้า สินค้า และคำสั่งซื้อตามสิทธิ์ของบทบาท

3.2.4.3 ลูกค้า ส่งข้อความและรับคำตอบจากเจ้าหน้าที่หรือแชทบอท

3.3 การออกแบบระบบ

3.3.1 สถาปัตยกรรมของระบบ แบ่งการทำงานออกเป็น 4 ส่วน ดังรูปที่ 3.2

[ใส่รูป: สถาปัตยกรรมของระบบ Chatshop]

รูปที่ 3.2 สถาปัตยกรรมของระบบ Chatshop

จากรูปที่ 3.2 ส่วนประกอบของระบบทำงาน ดังนี้

3.3.1.1 แอปพลิเคชันบนสมาร์ตโฟน พัฒนาด้วย Flutter เรียกข้อมูลผ่าน REST API รับเหตุการณ์ผ่าน WebSocket และรับการแจ้งเตือนผ่าน Firebase Cloud Messaging

3.3.1.2 ระบบบริการหลังบ้าน พัฒนาด้วย Laravel 10 รับ Webhook ประมวลผลข้อมูลในคิวงาน สร้างคำตอบของแชทบอท และเปิด REST API ให้แอปพลิเคชัน

3.3.1.3 ฐานข้อมูล ใช้ MySQL เก็บข้อมูลเชิงสัมพันธ์ ส่วนบทสนทนา ข้อความ ฐานความรู้ และโทเคนของอุปกรณ์เก็บใน MongoDB

3.3.1.4 บริการภายนอก ได้แก่ LINE Messaging API, Meta Graph API, บริการแบบจำลองภาษา และ Firebase Cloud Messaging

3.3.2 การออกแบบฐานข้อมูล ระบบแบ่งข้อมูลไว้ใน 2 ฐานข้อมูลตามลักษณะของข้อมูล แสดงความสัมพันธ์ดังรูปที่ 3.3

[ใส่รูป: ความสัมพันธ์ของข้อมูลใน MySQL และ MongoDB]

รูปที่ 3.3 ความสัมพันธ์ของข้อมูลใน MySQL และ MongoDB

จากรูปที่ 3.3 ข้อมูลที่ต้องการความถูกต้องและมีความสัมพันธ์กันชัดเจน เช่น ร้านค้า ทีม สินค้า และคำสั่งซื้อ เก็บใน MySQL ส่วนข้อมูลที่เพิ่มขึ้นตลอดเวลาและมีรูปแบบหลากหลาย เช่น บทสนทนาและข้อความ เก็บใน MongoDB ตารางและคอลเลกชันที่ใช้ในระบบมีหน้าที่ ดังนี้

ตารางที่ 3.1 ตารางในฐานข้อมูล MySQL

ชื่อตาราง	หน้าที่
tenants	เก็บข้อมูลร้านค้าที่ใช้งานระบบ และค่าตั้งต้นของร้าน เช่น ภาษา และเวลาที่ถือว่าแชทค้างตอบ
tenant_users	เก็บสมาชิกของร้าน ใช้ยืนยันตัวตนและระบุบทบาทของแต่ละคน
tenant_roles	เก็บบทบาทที่ร้านกำหนดเองและสิทธิ์ของแต่ละบทบาท
tenant_user_invitations	เก็บคำเชิญเข้าร่วมทีมที่ส่งทางอีเมลและสถานะการตอบรับ
personal_access_tokens	เก็บโทเคนที่ระบบออกให้แอปพลิเคชันหลังเข้าสู่ระบบ
businesses	เก็บธุรกิจภายใต้ร้านค้า พร้อมชื่อแบรนด์ ข้อมูลรับชำระเงิน และการตั้งค่าการทำงาน
platform_credentials	เก็บช่องทางที่ร้านเชื่อมต่อ และข้อมูลที่ใช้รับส่งข้อความกับแพลตฟอร์ม
business_chatbot_settings	เก็บการตั้งค่าแชทบอทของแต่ละธุรกิจ เช่น โหมด บุคลิก และแบบจำลองภาษาที่ใช้

ชื่อตาราง	หน้าที่
chatbot_directives	เก็บคำสั่งเพิ่มเติมที่ร้านกำหนดให้แชทบอทปฏิบัติ
customers	เก็บข้อมูลลูกค้าจากทุกช่องทาง เช่น ชื่อ เบอร์โทรศัพท์ และที่อยู่จัดส่ง
products	เก็บแคตตาล็อกสินค้าและราคาของแต่ละธุรกิจ
chat_orders	เก็บคำสั่งซื้อที่เปิดจากบทสนทนา พร้อมยอดรวมและสถานะ
tenant_tags	เก็บแท็กและสีที่ร้านใช้จัดกลุ่มบทสนทนา
saved_replies	เก็บข้อความสำเร็จรูปที่เจ้าหน้าที่เลือกส่งได้ทันที
automation_rules	เก็บกฎตอบกลับอัตโนมัติตามคีย์เวิร์ด
conversation_assignments	เก็บข้อมูลว่าบทสนทนาใดมอบหมายให้สมาชิกคนใด
import_runs	เก็บประวัติและสถานะการนำเข้าเซพย้อนหลัง

ตารางที่ 3.2 คอลเลกชันในฐานข้อมูล MongoDB

ชื่อคอลเลกชัน	หน้าที่
conversations	เก็บบทสนทนาของลูกค้าแต่ละรายในแต่ละช่องทาง พร้อมสถานะ เช่น ผู้รับผิดชอบ แท็ก จำนวนข้อความที่ยังไม่อ่าน และโหมดแชทบอท
messages	เก็บข้อความขาเข้าและขาออกทุกข้อความ ทั้งข้อความ รูปภาพ วิดีโอ ไฟล์ และสติ๊กเกอร์
knowledge_documents	เก็บเอกสารความรู้ที่ร้านเตรียมไว้ให้แชทบอทใช้ตอบคำถาม
knowledge_chunks	เก็บเอกสารความรู้ที่แบ่งเป็นส่วนย่อยพร้อมเวกเตอร์สำหรับค้นคืน
device_tokens	เก็บโทเคนของอุปกรณ์ที่ใช้ส่งการแจ้งเตือนแบบพุช

3.3.2.1 ความสัมพันธ์ระหว่างข้อมูล ร้านค้าหนึ่งแห่งมีสมาชิก บทบาท ธุรกิจ ลูกค้า และแท็กได้หลายรายการ ธุรกิจหนึ่งแห่งมีช่องทาง สินค้า และกฎตอบกลับได้หลายรายการ และมีการตั้งค่าแชทบอท 1 ชุด ลูกค้าหนึ่งรายมีบทสนทนาและคำสั่งซื้อได้หลายรายการ บทสนทนาหนึ่งรายการมีข้อความได้หลายข้อความ ส่วนเอกสารความรู้หนึ่งฉบับแบ่งเป็นส่วนย่อยได้หลายส่วน

3.3.2.2 การเชื่อมข้อมูลข้ามฐานข้อมูล ข้อมูลใน MongoDB อ้างอิงข้อมูลใน MySQL ด้วยรหัส เช่น บทสนทนาอ้างอิงร้านค้า ธุรกิจ และลูกค้า ส่วนคำสั่งซื้อและผู้รับผิดชอบบทสนทนาใน MySQL อ้างอิงบทสนทนาใน MongoDB ระบบหลังบ้านตรวจสอบความถูกต้องของการอ้างอิงเหล่านี้เอง เพราะฐานข้อมูลทั้งสองไม่บังคับความสัมพันธ์ข้ามกัน

3.3.3 ขั้นตอนการรับข้อความ แสดงดังรูปที่ 3.4

[ใส่รูป: ขั้นตอนการรับข้อความจาก Webhook]
รูปที่ 3.4 ขั้นตอนการรับข้อความจาก Webhook

จากรูปที่ 3.4 แพลตฟอร์มส่ง Webhook มายังระบบหลังบ้าน ระบบตรวจ
ลายเซ็นแล้วส่งงานเข้าคิว high และตอบกลับแพลตฟอร์มทันที จากนั้นงานในคิวจะบันทึกข้อมูล
ลูกค้าลง MySQL บันทึกบทสนทนาและข้อความลง MongoDB คัดลอกไฟล์สื่อมาเก็บในระบบ ส่ง
เหตุการณ์ไปยังแอปพลิเคชันผ่าน Laravel Reverb ส่งการแจ้งเตือนผ่าน Firebase Cloud
Messaging และส่งข้อความต่อให้กฎตอบกลับอัตโนมัติและแชทบอท

3.3.4 ขั้นตอนการทำงานของแชทบอท แสดงดังรูปที่ 3.5

[ใส่รูป: ขั้นตอนการสร้างคำตอบของแชทบอท]
รูปที่ 3.5 ขั้นตอนการสร้างคำตอบของแชทบอท

จากรูปที่ 3.5 ระบบรอ 6 วินาทีเพื่อรวมข้อความที่ลูกค้าส่งติดกัน แล้วค้นคืน
ส่วนย่อยของเอกสารความรู้ที่เกี่ยวข้อง ประกอบข้อความสั่งงานจากบุคลิกของร้าน ข้อมูลแบรนด์
บทสรุป และข้อความล่าสุด 20 ข้อความ ส่งให้แบบจำลองภาษาสร้างคำตอบ หากแบบจำลองตอบว่า
ต้องส่งต่อ ระบบจะติดแท็กรอแอดมินและเปลี่ยนเป็นโหมด Manual

3.3.5 ขั้นตอนการรับข้อมูลแบบเรียลไทม์และการแจ้งเตือน แสดงดังรูปที่ 3.6

[ใส่รูป: ขั้นตอนการรับเหตุการณ์และการแจ้งเตือนบนแอปพลิเคชัน]
รูปที่ 3.6 ขั้นตอนการรับเหตุการณ์และการแจ้งเตือนบนแอปพลิเคชัน

จากรูปที่ 3.6 เมื่อเข้าสู่ระบบ แอปพลิเคชันขออนุญาตแสดงการแจ้งเตือน
ลงทะเบียนโทเคนของอุปกรณ์กับระบบหลังบ้าน และเชื่อมต่อช่องส่วนตัวของร้านค้าผ่าน Laravel
Reverb เมื่อได้รับเหตุการณ์ แอปพลิเคชันจะทำงาน ดังนี้

3.3.5.1 MessageReceived เพิ่มข้อความในหน้าต่างสนทนาที่เปิดอยู่ และปรับ
ข้อความล่าสุดกับจำนวนที่ยังไม่อ่านในกล่องข้อความ

3.3.5.2 ConversationUpdated ปรับข้อมูลของบทสนทนา เช่น ผู้รับผิดชอบ
แท็ก และโหมดแชทบอท

3.3.5.3 NotificationCreated ดึงรายการแจ้งเตือนใหม่

3.3.5.4 OrderUpdated โหลดคำสั่งซื้อของบทสนทนาและรายการคำสั่งซื้อใหม่

3.3.5.5 SettingsChanged โหลดการตั้งค่าของร้านค้าใหม่

3.3.6 การแจ้งเตือนแบบพุช หากแอปพลิเคชันไม่ได้เปิดอยู่ ผู้ใช้จะได้รับการแจ้งเตือนจาก Firebase Cloud Messaging เมื่อแต่ละการแจ้งเตือน แอปพลิเคชันจะเปิดหน้าต่างสนทนานั้น หรือเปิดหน้ารายการคำสั่งซื้อของบทสนทนาเมื่อเป็นการแจ้งชำระเงิน ส่วนขณะเปิดแอปพลิเคชันอยู่ จะแสดงแถบแจ้งเตือนด้านล่างพร้อมปุ่มเปิดบทสนทนา และไม่แสดงเมื่อผู้ใช้อยู่ในบทสนทนานั้นแล้ว

3.4 โครงสร้างแอปพลิเคชันบนสมาร์ทโฟน

ผู้จัดทำแบ่งโค้ดของแอปพลิเคชันเป็น 4 ส่วนตามหน้าที่ ดังรูปที่ 3.7

ใส่รูป: โครงสร้างโค้ดของแอปพลิเคชัน
รูปที่ 3.7 โครงสร้างโค้ดของแอปพลิเคชัน

จากรูปที่ 3.7 โค้ดแต่ละส่วนทำหน้าที่ ดังนี้

3.4.1 ส่วน core เป็นบริการพื้นฐานที่ทุกหน้าจอใช้ร่วมกัน ประกอบด้วย flutter_secure_storage สำหรับเก็บโทเคน ตัวเรียก REST API ที่แนบโทเคนให้ทุกคำขอและส่งไฟล์แบบ multipart ได้ บริการเรียลไทม์ บริการแจ้งเตือน และ go_router

3.4.2 ส่วน data เป็นแบบจำลองข้อมูล เช่น บทสนทนา ข้อความ และการแจ้งเตือน และ Repository ที่แปลงข้อมูล JSON จากระบบหลังบ้านเป็นแบบจำลองข้อมูลของแอปพลิเคชัน

3.4.3 ส่วน features เป็นหน้าจอและตรรกะของแต่ละโมดูล 13 โมดูล ได้แก่ การยืนยันตัวตน กล่องข้อความ หน้าต่างสนทนา การค้นหา การแจ้งเตือน ลูกค้า สินค้า คำสั่งซื้อ ทิม รายงานช่องทาง ข้อมูลผู้ใช้ และการตั้งค่า

3.4.4 ส่วน shared เป็นวิดเจ็ตที่หลายหน้าจอใช้ร่วมกัน เช่น แถบนำทางด้านล่าง และแผ่นฟอร์มที่เลื่อนขึ้นจากด้านล่างของหน้าจอ

3.4.5 การจัดการสถานะและเส้นทางหน้าจอ การจัดการสถานะใช้ Riverpod โดยสถานะการเข้าสู่ระบบเป็น Provider ต้นทาง เมื่อผู้ใช้ออกจากระบบ Provider ของตัวเรียก API บริการเรียลไทม์ และบริการแจ้งเตือนจะยกเลิกการเชื่อมต่อและล้างข้อมูลของผู้ใช้เดิมทั้งหมด การกำหนดเส้นทางแบ่งเป็นหน้าเข้าสู่ระบบ หน้าสมัครใช้งาน และแถบนำทางด้านล่าง 3 แท็บ ได้แก่ กล่องข้อความ การแจ้งเตือน และข้อมูลผู้ใช้ ซึ่งรวมเมนูไปยังหน้าจอจัดการอื่นทั้งหมด หากได้รับรหัส 401 จากระบบหลังบ้าน แอปพลิเคชันจะลบโทเคนและกลับไปหน้าเข้าสู่ระบบทันที

3.5 โครงสร้างระบบบริการหลังบ้าน

ระบบบริการหลังบ้านพัฒนาด้วย Laravel 10 ประกอบด้วยส่วนต่าง ๆ ดังนี้

- 3.5.1 ส่วนรับและส่งข้อความ ประกอบด้วย ตัวรับ Webhook ของ LINE และ Meta ที่ตรวจสอบลายเซ็นด้วย HMAC-SHA256 และบริการส่งข้อความออกผ่าน LINE Push API และ Meta Graph API เวอร์ชัน 19.0
- 3.5.2 ส่วนคิวงานและตัวจัดตารางเวลา ใช้ประมวลผล Webhook แชนบอท และการนำเข้าประวัติแชทในเบื้องหลัง และตรวจหาแชทที่ค้างตอบทุก 1 นาที
- 3.5.3 ส่วนเรียลไทม์และการแจ้งเตือน ใช้ Laravel Reverb เป็นเซิร์ฟเวอร์ WebSocket และส่งการแจ้งเตือนไปยังโทเคนของอุปกรณ์ผ่าน Firebase Cloud Messaging
- 3.5.4 ส่วนยืนยันตัวตน ใช้ Laravel Sanctum ออกโทเคนให้แอปพลิเคชัน และตรวจสอบโทเคนของ Facebook และ Google ที่แอปพลิเคชันส่งมาก่อนออกโทเคนของระบบ
- 3.5.5 ส่วนจัดการไฟล์สื่อ คัดลอกไฟล์สื่อเข้ามาเก็บในระบบ ย่อรูปภาพด้วย PHP GD และย่อวิดีโอด้วย ffmpeg ก่อนจัดเก็บ
- 3.5.6 ส่วนแชทบอท ประกอบข้อความสั่งงาน ค้นคืนเอกสารความรู้ และเรียกบริการแบบจำลองภาษา ได้แก่ Ollama, Anthropic Claude, Google Gemini และบริการที่รองรับรูปแบบ OpenAI API
- 3.5.7 ส่วนการติดตั้งระบบ [รอข้อมูล: เครื่องแม่ข่ายที่ใช้จริง โดเมน และการตั้งค่า Queue worker และ Scheduler บนเครื่องแม่ข่าย]

3.6 การออกแบบ API

ระบบหลังบ้านเปิด REST API ให้แอปพลิเคชันเรียกใช้ประมาณ 99 เส้นทาง แบ่งเป็นหมวดตามหน้าที่ ทุกเส้นทางยกเว้นการสมัครและเข้าสู่ระบบต้องแนบโทเคนที่ได้รับหลังเข้าสู่ระบบ และระบบตรวจสอบสิทธิ์ตามบทบาทของผู้ใช้ก่อนทำงาน ข้อมูลที่รับส่งใช้รูปแบบ JSON หมวดของ API มีดังนี้

ตารางที่ 3.3 หมวดของ API

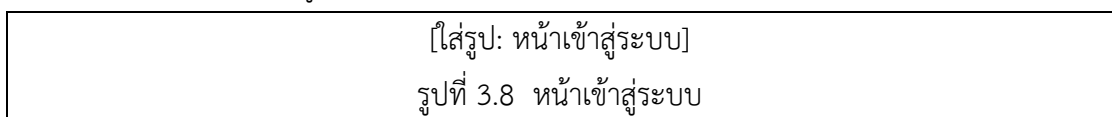
หมวด	หน้าที่
การยืนยันตัวตน	สมัครใช้งานและสร้างร้านค้าใหม่ เข้าสู่ระบบด้วยอีเมลหรือบัญชี Facebook และ Google ออกจากระบบ และดูข้อมูลผู้ใช้
บทสนทนา	ดูและค้นหาบทสนทนา ดูข้อความ ตอบข้อความและแนบไฟล์ มอบหมาย ติดแท็ก ปักหมุด จัดเก็บ และล๊อคบทสนทนา
แชทบอท	เปลี่ยนโหมดแชทบอทของบทสนทนา ขอคำตอบแนะนำ และดูผลวิเคราะห์บทสนทนา
ลูกค้า	ดู ค้นหา และแก้ไขข้อมูลลูกค้า

หมวด	หน้าที่
สินค้า	ดู เพิ่ม แก้ไข และลบสินค้า
คำสั่งซื้อ	เปิดคำสั่งซื้อจากบทสนทนา ดูรายการคำสั่งซื้อ และเปลี่ยนสถานะคำสั่งซื้อ
ทีม	ดูรายชื่อสมาชิก ส่งคำเชิญ เปลี่ยนบทบาท นำสมาชิกออก และโอนความเป็นเจ้าของร้าน
ช่องทาง	เชื่อมต่อ Facebook Page, Instagram, LINE Official Account และ WebChat และนำเข้าแชทย้อนหลัง
การแจ้งเตือนและอุปกรณ์	ดูและอ่านการแจ้งเตือน ลงทะเบียนและยกเลิกโทเคนของอุปกรณ์
รายงาน	ดูตัวชี้วัดการให้บริการและสัดส่วนข้อความของแต่ละช่องทางตามช่วงเวลา
การตั้งค่า	ตั้งค่าร้านค้า ธุรกิจ แท็ก แหทบอท และการแจ้งเตือน

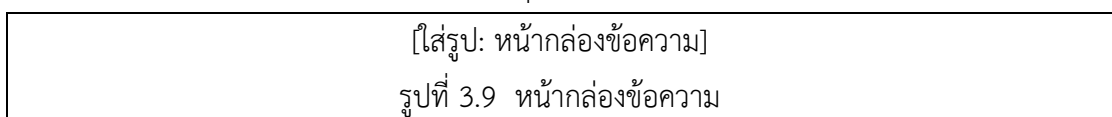
3.7 การพัฒนาแอปพลิเคชันบนสมาร์ตโฟน

ผู้จัดทำพัฒนาหน้าจอของแอปพลิเคชันตามความต้องการในหัวข้อ 3.2 ดังนี้

3.7.1 หน้าเข้าสู่ระบบและสมัครใช้งาน ผู้ใช้เข้าสู่ระบบด้วยอีเมลและรหัสผ่าน หรือตะปุมเข้าสู่ระบบด้วยบัญชี Facebook หรือ Google ซึ่งเปิดหน้าเข้าสู่ระบบของผู้ให้บริการโดยตรง หน้าสมัครใช้งานตรวจสอบรูปแบบอีเมลและกำหนดรหัสผ่านอย่างน้อย 8 ตัวอักษร



3.7.2 หน้ากล่องข้อความ แสดงบทสนทนาจากทุกช่องทางโดยเรียงบทสนทนาที่ปิดหมดไว้ก่อน คัดกรองได้ตามข้อความที่ยังไม่อ่าน บทสนทนาที่ปิดหมด และช่องทาง เลือกแสดงเฉพาะเพจที่ต้องการได้ และค้นหาบทสนทนาของทั้งร้านค้าจากชื่อ ข้อความ และแท็ก ผู้ใช้ปัดรายการเพื่อจัดเก็บหรือลบบทสนทนา และกดค้างเพื่อปิดหมดหรือมอบหมายบทสนทนาให้สมาชิกในทีม



3.7.3 หน้าต่างสนทนา แสดงข้อความของบทสนทนาและโหลดข้อความเก่าเมื่อเลื่อนขึ้น เจ้าหน้าที่ตอบข้อความ แบนรูปภาพ วิดีโอ และไฟล์เอกสาร เลือกข้อความสำเร็จรูป และกดค้างที่

ข้อความเพื่อตอบกลับโดยอ้างอิง ปักหมุด แสดงความรู้สึก หรือใช้ข้อความนั้นเป็นที่ยู่อจัดส่ง นอกจากนี้ยังค้นหาข้อความภายในบทสนทนาได้ หากร้านเปิดการลือกบทสนทนาและมีผู้อื่นรับผิดชอบอยู่ แอปพลิเคชันจะขอให้ยืนยันก่อนตอบ

[ใส่รูป: หน้าต่างสนทนา]

รูปที่ 3.10 หน้าต่างสนทนา

3.7.4 การควบคุมแชทบอทและการวิเคราะห์บทสนทนา เจ้าหน้าที่เลือกโหมดแชทบอทของบทสนทนาได้ 3 โหมด ในโหมด Suggest แอปพลิเคชันแสดงคำตอบแนะนำเหนือช่องพิมพ์ ในโหมด Autopilot แอปพลิเคชันซ่อนช่องพิมพ์และแสดงว่าแชทบอทกำลังตอบแทน ส่วนแผ่นวิเคราะห์บทสนทนาแสดงบทสรุป ความรู้สึก เจตนา หัวข้อที่ลูกค้าสนใจ และสัญญาณสำคัญ เช่น ลูกค้าประจำหรือเรื่องเร่งด่วน

[ใส่รูป: แผ่นวิเคราะห์บทสนทนา]

รูปที่ 3.11 แผ่นวิเคราะห์บทสนทนา

3.7.5 ข้อมูลลูกค้าและการเปิดคำสั่งซื้อ แผ่นข้อมูลลูกค้าในหน้าต่างสนทนาแสดงช่องทางที่อยู่ บันทึกภายใน แท็ก และประวัติคำสั่งซื้อ แบบฟอร์มคำสั่งซื้อช่วยเติมซื้อสินค้าจากแคตตาล็อกคำนวณยอดรวมจากราคาและจำนวนทันที และบันทึกที่อยู่จัดส่ง

[ใส่รูป: แบบฟอร์มเปิดคำสั่งซื้อ]

รูปที่ 3.12 แบบฟอร์มเปิดคำสั่งซื้อ

3.7.6 หน้าการแจ้งเตือน แสดงการแจ้งเตือน 5 ประเภท ได้แก่ เร่งด่วน ข้อความใหม่ ชำระเงินแล้ว การกล่าวถึง และระบบ ตะเพื่ออ่านและเปิดบทสนทนา และอ่านทั้งหมดได้

[ใส่รูป: หน้าการแจ้งเตือน]

รูปที่ 3.13 หน้าการแจ้งเตือน

3.7.7 หน้าลูกค้า สินค้า และคำสั่งซื้อ หน้าลูกค้าค้นหาและแก้ไขเบอร์โทรศัพท์ ที่อยู่ และบันทึกภายในได้ หน้าสินค้าเพิ่ม แก้ไข และลบสินค้าได้ตามสิทธิ์ หน้าคำสั่งซื้อแยกแท็บตามสถานะค้นหาได้ และโหลดรายการเพิ่มเมื่อเลื่อนลง

[ใส่รูป: หน้ารายการคำสั่งซื้อ]

รูปที่ 3.14 หน้ารายการคำสั่งซื้อ

3.7.8 หน้าจัดการทีม เจ้าของร้านเชิญสมาชิกทางอีเมลพร้อมบทบาท จัดการค่าเชิญที่ค้างอยู่ เปลี่ยนบทบาท นำสมาชิกออก และโอนความเป็นเจ้าของร้านโดยยืนยันด้วยรหัสผ่าน

[ใส่รูป: หน้าจัดการทีม]

รูปที่ 3.15 หน้าจัดการทีม

3.7.9 หน้ารายงาน แสดงตัวชี้วัดการให้บริการและสัดส่วนปริมาณข้อความของแต่ละช่องทาง เลือกช่วงเวลาได้ ได้แก่ วันนี้ 7 วัน 30 วัน และรายไตรมาส

[ใส่รูป: หน้ารายงาน]

รูปที่ 3.16 หน้ารายงาน

3.7.10 หน้าเชื่อมต่อช่องทาง เชื่อมต่อ Facebook Page และ Instagram ผ่านหน้าอนุญาตสิทธิ์ของ Facebook แล้วเลือกเพจที่ต้องการ เชื่อมต่อ LINE Official Account ด้วย Channel ID และ Channel Secret สร้างโค้ดสำหรับติด WebChat บนเว็บไซต์ และนำเข้าประวัติแชทย้อนหลังได้ไม่เกิน 90 วัน

[ใส่รูป: หน้าเชื่อมต่อช่องทาง]

รูปที่ 3.17 หน้าเชื่อมต่อช่องทาง

3.7.11 หน้าข้อมูลผู้ใช้และการตั้งค่า แสดงสถิติของผู้ใช้ เช่น จำนวนข้อความที่ตอบวันนี้ และยอดขายวันนี้ และเป็นเมนูไปยังหน้าตั้งค่าร้านค้า ธุรกิจ แท็ก การล๊อคบทสนทนาและโหมดเริ่มต้นของแชทบอท แบบจำลองภาษาของแชทบอท เวลาแจ้งเตือนแชทที่ค้างตอบ 5 10 15 หรือ 20 นาที และการแจ้งเตือนรายผู้ใช้

[ใส่รูป: หน้าข้อมูลผู้ใช้และเมนูการตั้งค่า]

รูปที่ 3.18 หน้าข้อมูลผู้ใช้และเมนูการตั้งค่า

3.8 การเชื่อมต่อกับ LINE Official Account และ Meta

3.8.1 การเชื่อมต่อ LINE Official Account ร้านค้านำ Channel ID และ Channel Secret จาก LINE Developers Console มากรอกในแอปพลิเคชัน แล้วนำ URL ของ Webhook ที่ระบบสร้างให้ไปตั้งค่าใน LINE Developers Console ระบบหลังบ้านใช้ Channel Secret ตรวจสอบลายเซ็นของทุก Webhook ที่ได้รับ และส่งข้อความตอบกลับผ่าน Push API

3.8.2 การเชื่อมต่อ Facebook Page และ Instagram แอปพลิเคชันเปิดหน้าอนุญาตสิทธิ์ของ Facebook ผ่าน OAuth เมื่อผู้ใช้อนุญาตแล้วจะกลับเข้าแอปพลิเคชันพร้อมรายการเพจหรือบัญชี Instagram ให้เลือก ระบบหลังบ้านบันทึก Page Access Token ไว้สำหรับรับส่งข้อความตรวจสอบลายเซ็น Webhook ด้วย App Secret หาก token หมดอายุ ระบบจะปิดช่องทางการนั้น

3.8.3 การเชื่อมต่อ WebChat ร้านค้าสร้างช่องทาง WebChat จากแอปพลิเคชัน แล้วนำโค้ดที่ระบบสร้างให้ไปติดบนเว็บไซต์ของร้าน